



Lab 1B - Complete Walkthrough & Incident Response Guide

This guide provides the **exact steps, CLI commands, and troubleshooting paths** for Lab 1B. Organized by phase with dependency mapping, common issues, and all three incident scenarios covered.

Pre-Stage: Confirm Lab 1A Completion



CRITICAL: Lab 1B builds on Lab 1A. You cannot proceed without these values confirmed.

Value Discovery Commands (Run These First)



Run these commands ONCE at the start. Copy the outputs into the table below so you have them for the entire lab.

```
# =====  
# ACCOUNT & REGION  
# =====  
  
# Get your AWS Account ID  
aws sts get-caller-identity --query "Account" --output text  
  
# Get your current region  
aws configure get region  
  
# =====  
# RDS VALUES  
# =====  
  
# Get RDS Endpoint  
aws rds describe-db-instances \  
  --db-instance-identifier lab-mysql \  
  --query "DBInstances[0].Endpoint.Address" --output text
```

```

# Get RDS Port
aws rds describe-db-instances \
  --db-instance-identifier lab-mysql \
  --query "DBInstances[0].Endpoint.Port" --output text

# Get RDS Security Group ID
aws rds describe-db-instances \
  --db-instance-identifier lab-mysql \
  --query "DBInstances[0].VpcSecurityGroups[0].VpcSecurityGroupId" --output
t text

# Get DB Name
aws rds describe-db-instances \
  --db-instance-identifier lab-mysql \
  --query "DBInstances[0].DBName" --output text

# =====
# EC2 VALUES
# =====

# Get EC2 Instance ID (filter by your instance name)
aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=*lab*" "Name=instance-state-name,Values=
running" \
  --query "Reservations[0].Instances[0].InstanceId" --output text

# Get EC2 Public IP
aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=*lab*" "Name=instance-state-name,Values=
running" \
  --query "Reservations[0].Instances[0].PublicIpAddress" --output text

# Get EC2 Security Group ID
aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=*lab*" "Name=instance-state-name,Values=
running" \
  --query "Reservations[0].Instances[0].SecurityGroups[0].GroupId" --output
t text

# Get EC2 IAM Role Name
aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=*lab*" "Name=instance-state-name,Values=

```

```
running" \
  --query "Reservations[0].Instances[0].IamInstanceProfile.Arn" --output t
ext | cut -d'/' -f2
```

Required Values from Lab 1A (Fill These In)

Resource	Example	Your Value
AWS Account ID	919113286081	
AWS Region	us-east-2	
RDS Endpoint	lab-mysql.chkce02amfxr.us-east-2.rds.amazonaws.com	
RDS Port	3306	
DB Name	labdb	
DB Username	admin	
DB Password	(your password)	
EC2 Public IP	3.15.xxx.xxx	
EC2 Instance Role	RDS-lab-Access	
EC2 Security Group ID	sg-012119988bf5e29f1	
RDS Security Group ID	sg-01f5bc3e6b016dbb2	

Quick Verification (Run Before Starting)

```
# Confirm EC2 → RDS connectivity works
curl http://<EC2_PUBLIC_IP>/list
```

Expected: Data returns successfully. If this fails, fix Lab 1A first.

Dependency Chain (Execute in This Order)

PHASE 1: DUAL SECRET STORAGE (Foundation for everything else)

1.1 Create SSM Parameter Store entries

→ /lab/db/endpoint, /lab/db/port, /lab/db/name

1.2 Create Secrets Manager secret

→ lab/rds/mysql (JSON: username, password, host, port)

1.3 Update IAM Role with SSM + Secrets Manager permissions

1.4 Verify EC2 can read BOTH systems



PHASE 2: OBSERVABILITY SETUP (Must complete before incident simulation)

2.1 Redirect Flask logs to file (/var/log/rdsapp/app.log)

2.2 Add ERROR logging to

PHASE 1: DUAL SECRET STORAGE

Phase 1 - Dual Secret Storage

PHASE 2: OBSERVABILITY SETUP

Phase 2 - Observability Setup

PHASE 3: ALARM CONFIGURATION

Phase 3 - Alarm Configuration

PHASE 4: INCIDENT SCENARIOS (ALL THREE)

Phase 4 - All Incident Scenarios

PHASE 5: INCIDENT RESPONSE & RECOVERY

Phase 5 - Incident Response & Recovery

PHASE 6: DELIVERABLES & REFLECTION

Phase 6 - Deliverables & Reflection

Quick Reference: Common Issues by Phase

Phase	Common Issue	Root Cause	Fix
1	<code>AccessDeniedException</code> on SSM	Missing <code>ssm:GetParameter</code> permission	Add inline policy to EC2 role
1	<code>AccessDeniedException</code> on Secrets Manager	Missing <code>secretsmanager:GetSecretValue</code> or KMS key issue	Check IAM policy ARN match + encryption key
2	CloudWatch Agent won't start	Config file syntax error or wrong path	Check <code>/opt/aws/amazon-cloudwatch-agent/logs/</code>
2	Log group doesn't appear	Missing <code>logs:CreateLogGroup</code> permission	Attach <code>CloudWatchAgentServerPolicy</code>
3	Can't create metric filter	No ERROR logs exist yet	Trigger an error first, then create filter
4	Alarm doesn't fire	App not logging "ERROR" keyword	Update <code>get_conn()</code> in <code>app.py</code>
5	App won't recover after fix	App caching old credentials	<code>sudo systemctl restart rdsapp</code>

Critical CLI Commands (Copy-Paste Ready)

Phase 1 Verifications

```
# Verify Parameter Store
aws ssm get-parameters \
  --names /lab/db/endpoint /lab/db/port /lab/db/name \
  --with-decryption

# Verify Secrets Manager
aws secretsmanager get-secret-value \
  --secret-id lab/rds/mysql
```

```
# Verify from EC2 (SSH in first)
aws ssm get-parameter --name /lab/db/endpoint
aws secretsmanager get-secret-value --secret-id lab/rds/mysql
```

Phase 2 Verifications

```
# Check log group exists
aws logs describe-log-groups \
  --log-group-name-prefix /aws/ec2/lab-rds-app

# Check CloudWatch Agent status (on EC2)
sudo systemctl status amazon-cloudwatch-agent

# View agent logs for errors (on EC2)
sudo tail -f /opt/aws/amazon-cloudwatch-agent/logs/amazon-cloudwatch-agent.log
```

Phase 3 Verifications

```
# Check alarm exists
aws cloudwatch describe-alarms \
  --alarm-name-prefix lab-db-connection
```

Phase 4 Incident Commands

```
# SCENARIO A: Block Security Group
aws ec2 revoke-security-group-ingress \
  --group-id <RDS_SG_ID> \
  --protocol tcp \
  --port 3306 \
  --source-group <EC2_SG_ID>

# SCENARIO B: Stop RDS
aws rds stop-db-instance --db-instance-identifier lab-mysql

# SCENARIO C: Change Password (breaks credentials)
aws secretsmanager update-secret \
  --secret-id lab/rds/mysql \
  --secret-string '{"username":"admin","password":"WRONGPASSWORD","host": "<RDS_ENDPOINT>","port":"3306"}'
```

```
# Trigger the error
curl http://<EC2_PUBLIC_IP>/list
```

Phase 5 Recovery Commands

```
# Check ERROR logs
aws logs filter-log-events \
  --log-group-name /aws/ec2/lab-rds-app \
  --filter-pattern "ERROR" \
  --limit 10

# Check alarm state
aws cloudwatch describe-alarms \
  --alarm-names lab-db-connection-failure-alarm \
  --query "MetricAlarms[0].StateValue"

# RESTORE A: Re-authorize Security Group
aws ec2 authorize-security-group-ingress \
  --group-id <RDS_SG_ID> \
  --protocol tcp \
  --port 3306 \
  --source-group <EC2_SG_ID>

# RESTORE B: Start RDS
aws rds start-db-instance --db-instance-identifier lab-mysql
aws rds wait db-instance-available --db-instance-identifier lab-mysql

# RESTORE C: Fix password in Secrets Manager
aws secretsmanager update-secret \
  --secret-id lab/rds/mysql \
  --secret-string '{"username":"admin","password":"<CORRECT_PASSWORD>","host":"<RDS_ENDPOINT>","port":"3306"}'
sudo systemctl restart rdsapp # On EC2

# Verify recovery
curl http://<EC2_PUBLIC_IP>/list
```

IAM Policies Reference (Copy-Paste Ready)

SSM Parameter Store Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadLabParameters",
      "Effect": "Allow",
      "Action": [
        "ssm:GetParameter",
        "ssm:GetParameters"
      ],
      "Resource": "arn:aws:ssm:<REGION>:<ACCOUNT_ID>:parameter/lab/db/*"
    }
  ]
}
```

Secrets Manager Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadSpecificSecret",
      "Effect": "Allow",
      "Action": ["secretsmanager:GetSecretValue"],
      "Resource": "arn:aws:secretsmanager:<REGION>:<ACCOUNT_ID>:secret:lab/rds/mysql*"
    }
  ]
}
```

CloudWatch Logs Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents",

```

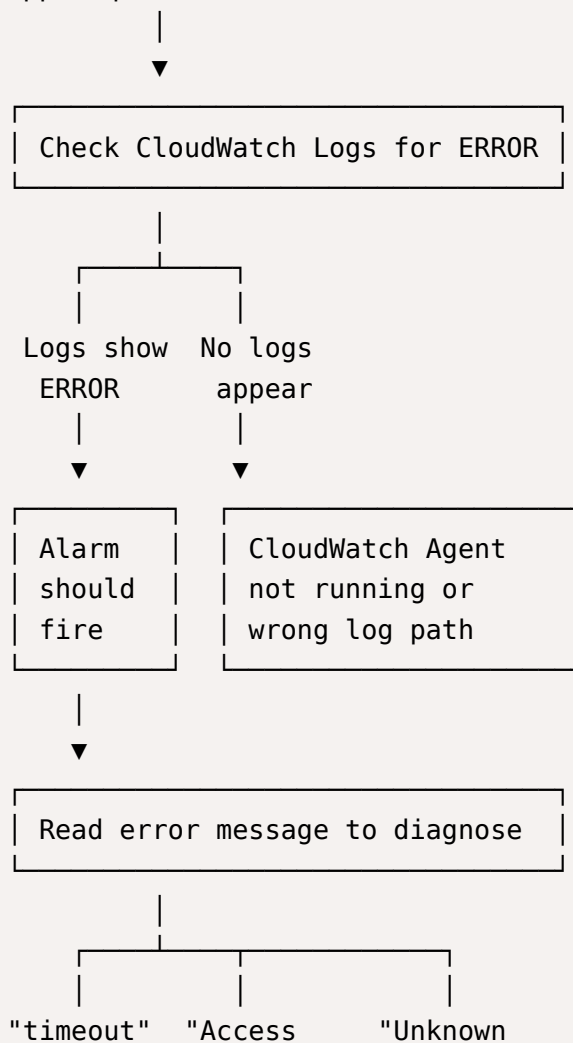
```

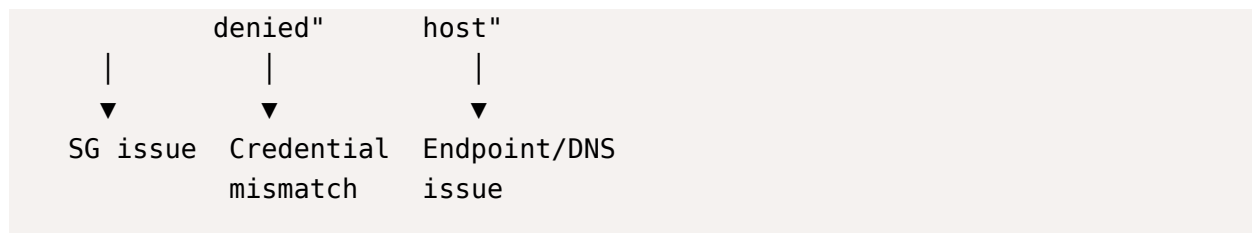
    "logs:DescribeLogStreams",
    "logs:DescribeLogGroups"
  ],
  "Resource": [
    "arn:aws:logs:<REGION>:<ACCOUNT_ID>:log-group:/aws/ec2/lab-rds-ap
p:*",
    "arn:aws:logs:<REGION>:<ACCOUNT_ID>:log-group:*"
  ]
}
]
}
}

```

Troubleshooting Decision Tree

App request fails after incident simulation





What Completing Lab 1B Proves



If you complete this lab, you can confidently say:

"I can operate, monitor, and recover AWS workloads using proper secret management and observability."

That is mid-level engineer capability, not entry-level.



Lab 1B Execution Log (January 2026)



Status: COMPLETE — Phases 1-2 executed. Phases 3-6 skipped (practiced via real incident response with group lead sabotage).

Actual Environment Values

Resource	Actual Value
AWS Account ID	919113286081
AWS Region	us-east-2
RDS Instance	helga-rds01
RDS Endpoint	helga-rds01.chkce02amfxr.us-east-2.rds.amazonaws.com
EC2 Instance	i-0b2ddc7d88b6b7bed
EC2 Role	helga-ec2-role01
Secret ID	lab/rds/mysql
Log Group	/aws/ec2/lab-rds-app
App Path	/opt/rdsapp/ app.py



Sabotage Attacks Encountered (Group Lead Exercise)

Group lead was given least-privilege IAM credentials and used them to inject configuration errors:

Attack	What Was Changed	Poisoned Value	Correct Value	Error Seen
Typo Attack	SSM /lab/db/endpoint	helga-rds01... (numeral 1)	helga-rds01... (letter l)	Unknown MySQL server host
Prefix Injection	SSM /lab/db/name	t_labdb	labdb	Unknown database 't_labdb'
Username Poison	Secrets Manager username	adm1n (numeral 1)	admin (letter i)	Access denied for user 'adm1n'

Detection Method: CloudWatch Logs showed exact error messages that didn't match expected values. Cross-referencing SSM/Secrets Manager values against actual RDS revealed the mismatches.

Lesson Learned: Visually similar character substitution (1/l, 1/i, 0/O) is a classic attack vector. Always verify values programmatically, not visually.

Fixes Applied

```
# Fix 1: Correct SSM endpoint typo
MSYS_NO_PATHCONV=1 aws ssm put-parameter \
  --name /lab/db/endpoint \
  --value "helga-rds01.chkce02amfxr.us-east-2.rds.amazonaws.com" \
  --type SecureString \
  --overwrite \
  --region us-east-2

# Fix 2: Correct SSM db name
MSYS_NO_PATHCONV=1 aws ssm put-parameter \
  --name /lab/db/name \
  --value "labdb" \
  --type String \
  --overwrite \
  --region us-east-2

# Fix 3: Update secret with correct username
MSYS_NO_PATHCONV=1 aws secretsmanager update-secret \
  --secret-id lab/rds/mysql \
  --secret-string '{"username":"admin","password":"<PASSWORD>","host":"helga-rds01.chkce02amfxr.us-east-2.rds.amazonaws.com","port":"3306"}' \
  --region us-east-2
```

Windows Git Bash Issue Encountered

Git Bash on Windows converts paths starting with `/` to Windows paths (e.g., `/lab/db/endpoint` → `C:/Program Files/Git/lab/db/endpoint`).

Fix: Prefix commands with `MSYS_NO_PATHCONV=1`:

```
MSYS_NO_PATHCONV=1 aws ssm get-parameters --names /lab/db/endpoint /lab/db/port /lab/db/name --with-decryption
```

✓ Verification Script (All 5 Checks Passed)

```
#!/bin/bash
REGION="us-east-2"
RDS_ID="helga-rds01"
PASS=0
FAIL=0

echo "🔍 Lab 1B Verification Starting..."
echo "===== "

# 1. RDS Status
RDS_STATUS=$(aws rds describe-db-instances --db-instance-identifier $RDS_ID --region $REGION --query "DBInstances[0].DBInstanceStatus" --output text 2>/dev/null)
if [ "$RDS_STATUS" == "available" ]; then
    echo "✓ RDS Status: $RDS_STATUS"
    ((PASS++))
else
    echo "✗ RDS Status: $RDS_STATUS (expected: available)"
    ((FAIL++))
fi

# 2. SSM Endpoint matches RDS
RDS_ENDPOINT=$(aws rds describe-db-instances --db-instance-identifier $RDS_ID --region $REGION --query "DBInstances[0].Endpoint.Address" --output text)
SSM_ENDPOINT=$(MSYS_NO_PATHCONV=1 aws ssm get-parameter --name /lab/db/endpoint --with-decryption --region $REGION --query "Parameter.Value" --output text)
if [ "$RDS_ENDPOINT" == "$SSM_ENDPOINT" ]; then
    echo "✓ SSM endpoint matches RDS"
    ((PASS++))
else
```

```

    echo "❌ SSM endpoint MISMATCH"
    ((FAIL++))
fi

# 3. Secret host matches RDS
SECRET_HOST=$(MSYS_NO_PATHCONV=1 aws secretsmanager get-secret-value --secret-id lab/rds/mysql --region $REGION --query "SecretString" --output text | jq -r '.host')
if [ "$RDS_ENDPOINT" == "$SECRET_HOST" ]; then
    echo "✅ Secret host matches RDS"
    ((PASS++))
else
    echo "❌ Secret host MISMATCH"
    ((FAIL++))
fi

# 4. SSM port is 3306
SSM_PORT=$(MSYS_NO_PATHCONV=1 aws ssm get-parameter --name /lab/db/port --region $REGION --query "Parameter.Value" --output text)
if [ "$SSM_PORT" == "3306" ]; then
    echo "✅ SSM port: 3306"
    ((PASS++))
else
    echo "❌ SSM port: $SSM_PORT (expected: 3306)"
    ((FAIL++))
fi

# 5. SSM db name is labdb
SSM_NAME=$(MSYS_NO_PATHCONV=1 aws ssm get-parameter --name /lab/db/name --region $REGION --query "Parameter.Value" --output text)
if [ "$SSM_NAME" == "labdb" ]; then
    echo "✅ SSM db name: labdb"
    ((PASS++))
else
    echo "❌ SSM db name: $SSM_NAME (expected: labdb)"
    ((FAIL++))
fi

echo "====="
echo "Results: $PASS passed, $FAIL failed"

```

Result: 5/5 checks passed after fixes applied.

Final Outcome

Phase	Status	Notes
Phase 1 (Dual Secret Storage)	✓ Complete	SSM + Secrets Manager configured and verified
Phase 2 (Observability)	✓ Complete	CloudWatch logs flowing, ERROR logging added
Phase 3 (Alarms)	⏮ Skipped	Already practiced with group
Phase 4 (Incident Scenarios)	⏮ Skipped	Got real incident response from sabotage
Phase 5 (Recovery)	⏮ Skipped	Performed actual recovery during sabotage fix
Phase 6 (Deliverables)	⏮ Skipped	Documentation captured here

Group lead access keys: DEACTIVATED after sabotage exercise completed.